

PROYECTO INTEGRADOR

LÓGICA DE PROGRAMACIÓN

AUTOR: ANTHONY CONLAGO

DOCENTE: MSC. LILIAN AMAN



Contenido

- 01 Introducción
- 02 Descripción del problema
- 03 Relación con los contenidos de la asignatura
- 04 Cronograma
- 05 Explicación del sistema desarrollado
- 06 Reflexión sobre el impacto de la tecnología

Introducción

El presente documento describe el desarrollo de un programa en Python en el que la maquina adivina un numero pensado por el usuario. A diferencia de los juegos tradicionales donde es el humano quien adivina, en este proyecto es el computador quien aplica un proceso matematico sistematico para determinar el numero secreto del usuario en el menor numero de intentos posible.

El programa solicita al usuario que piense un numero entre 1 y 100, y mediante preguntas sucesivas de mayor o menor, va reduciendo el rango de busqueda hasta acertar. Este enfoque pone en practica conceptos fundamentales de la logica de programacion aprendidos a lo largo de la asignatura, demostrando como los algoritmos pueden resolver problemas de forma eficiente y estructurada.



Descripcion del Problema

EL PROBLEMA CONSISTE EN DISEÑAR UN ALGORITMO CAPAZ DE ADIVINAR CUALQUIER NUMERO ENTERO ENTRE 1 Y 100 QUE EL USUARIO TENGA EN MENTE, UTILIZANDO LA MENOR CANTIDAD DE INTENTOS POSIBLE Y SIN CONOCER EL NUMERO DE ANTEMANO.

PLANTEAMIENTO

DADO UN RANGO DE NUMEROS ENTRE 1 Y 100, EL PROGRAMA DEBE:

- PROPONER UN NUMERO CANDIDATO DE FORMA INTELIGENTE EN CADA INTENTO.
- RECIBIR RETROALIMENTACION DEL USUARIO: SI EL NUMERO PROPUESTO ES CORRECTO, MAYOR O MENOR AL PENSADO.
- AJUSTAR EL RANGO DE BUSQUEDA SEGUN LA RESPUESTA Y REPETIR EL PROCESO HASTA ACERTAR.
- INFORMAR AL USUARIO CUANTOS INTENTOS NECESITO PARA ADIVINAR EL NUMERO.

RESTRICCIONES DEL PROBLEMA

- EL NUMERO DEBE ESTAR EN EL RANGO DEL 1 AL 100.
- EL USUARIO SOLO PUEDE RESPONDER CON TRES OPCIONES: 1 (ACERTO), 2 (EL NUMERO ES MENOR) O 3 (EL NUMERO ES MAYOR).
- EL PROGRAMA DEBE TERMINAR CUANDO LA MAQUINA ACIERTE EL NUMERO.

POR QUE ES EFICIENTE?

MATEMATICAMENTE, CON BUSQUEDA BINARIA EL COMPUTADOR NECESITA A LO SUMO $\log_2(100) = 6.64$, ES DECIR UN MAXIMO DE 7 INTENTOS PARA ADIVINAR CUALQUIER NUMERO ENTRE 1 Y 100, INDEPENDIENTEMENTE DE CUAL SEA. ESTO DEMUESTRA LA SUPERIORIDAD DEL ALGORITMO FRENTE A UNA BUSQUEDA ALEATORIA O SECUENCIAL.

}

Relacion con los Contenidos de la Asignatura

El proyecto integra de forma practica los temas estudiados a lo largo de la asignatura de Logica de Programacion:

Introduccion a la Resolucion de Problemas

El juego en si es un problema bien definido: adivinar un numero con la menor cantidad de pasos. Se identificaron entradas, salidas y el proceso logico necesario.

Entorno de Programacion

El programa fue desarrollado en Python usando VS Code, aplicando buenas practicas de escritura de codigo limpio y legible.

Manejo de Datos y Algoritmos

Se manejan variables enteras (inferior, superior, intento, intentos) y se aplica el algoritmo de busqueda binaria como proceso central.

Algoritmos y Diagramas de Flujo

El flujo del programa sigue una estructura algoritmica clara: inicializar rango, calcular punto medio, recibir respuesta y ajustar limites.

Logica de Programacion

Se aplican estructuras condicionales (if / elif) para evaluar la respuesta del usuario y tomar decisiones en cada iteracion.

Bucles

El ciclo while True mantiene el juego activo hasta que la maquina acierta, usando break para terminar cuando la respuesta es correcta.

Manejo de Datos y Algoritmos

Se utilizan variables numericas para almacenar el estado del juego (limites del rango, contador de intentos) y la funcion `math.floor()` para calcular el punto medio.

}

CRONOGRAMA

SEMANA 1

SEMANA 2

SEMANA 3

SEMANA 4

SEMANA 5

SEMANA 6

SEMANA 7

SEMANA 8

UNIDAD 1

EN EL TRANCURSO DE LA UNIDAD VIMOS LOS TEMAS DE INTRODUCCION A LA RESOLUCION DE PROBLEMAS Y ENTORNO DE PROGRAMACION TAMBIEN SELECCIONAMOS EL PROGRAMA A DESARROLLAR Y ANALIZAMOS TODO EN RESPECTO AL JUEGO ESCOGIDO.

POSTERIOREMENTE EMPEZAMOS A TENER UNA IDEA DE QUE DEBE REALIZAR NUESTRO JUEGO Y QUE PROBLEMAS SE SOLUCIONARAN

UNIDAD 2

EN ESTA ETAPA EMPEZAMOS EL INICIO DEL DESARROLLO DEL PROGRAMA Y ANALIZAMOS LOS DIAGRAMAS DE FLUJO PARA PODER ENTENDER EL WNTORNO Y TODAS LAS SITUACIONES ALREDEDOR DE LA PROGRAMACION DEL JUEGO SELLECCIONADO

UNIDAD 3

EMPEZAMOS A DESARROLLAR EL PROGRAMA SELLECCIONADO A TRAVEZ DE LAS APLICACIONES RAPTOR PARA LOS DIAGRAMAS DE FUNCIONES Y LA APLICAION VISUAL ESTUDIO CODE, EMPEZAMOS REALIZANDO LAS PRIMERAS LINEAS DE CODIGO

UNIDAD 4

EN ESTA SEMANA DEBEMOS INCORPORAR LA INFORMACION RECIBIDA EN CLASES PARA SEGUIR ADELANTE TALES COMO BUCLES PARTE FUNDAMENTAL DEL PROYECTO ESCOGIDO, TAMBIEN SE DEPURA EL CODIGO Y SE COMPRUEBA LA FUNCIONALIDAD DEL MISMO.

}

Explicacion del Sistema Desarrollado

Logica general del algoritmo

EL SISTEMA IMPLEMENTA EL ALGORITMO DE BUSQUEDA BINARIA APLICADO A LA ADIVINANZA DE NUMEROS. EL PROCESO FUNCIONA DE LA SIGUIENTE MANERA:



- Se establece un rango inicial: `inferior = 1`, `superior = 100`.
- En cada iteracion, el programa calcula el punto medio del rango: `intento = math.floor((inferior + superior) / 2)`.
- Si el usuario responde que la maquina acerto (opcion 1), el juego termina mostrando el numero de intentos.
- Si el usuario responde que su numero es menor (opcion 2), el limite superior se reduce: `superior = intento - 1`.
- Si el usuario responde que su numero es mayor (opcion 3), el limite inferior sube: `inferior = intento + 1`.
- El contador de intentos se incrementa en cada ciclo.

}

Codigo desarrollado (Python)

```
import math

print(" La maquina adivina tu numero ")
print("Piensa un numero del 1 al 100")

inferior = 1
superior = 100
intentos = 0

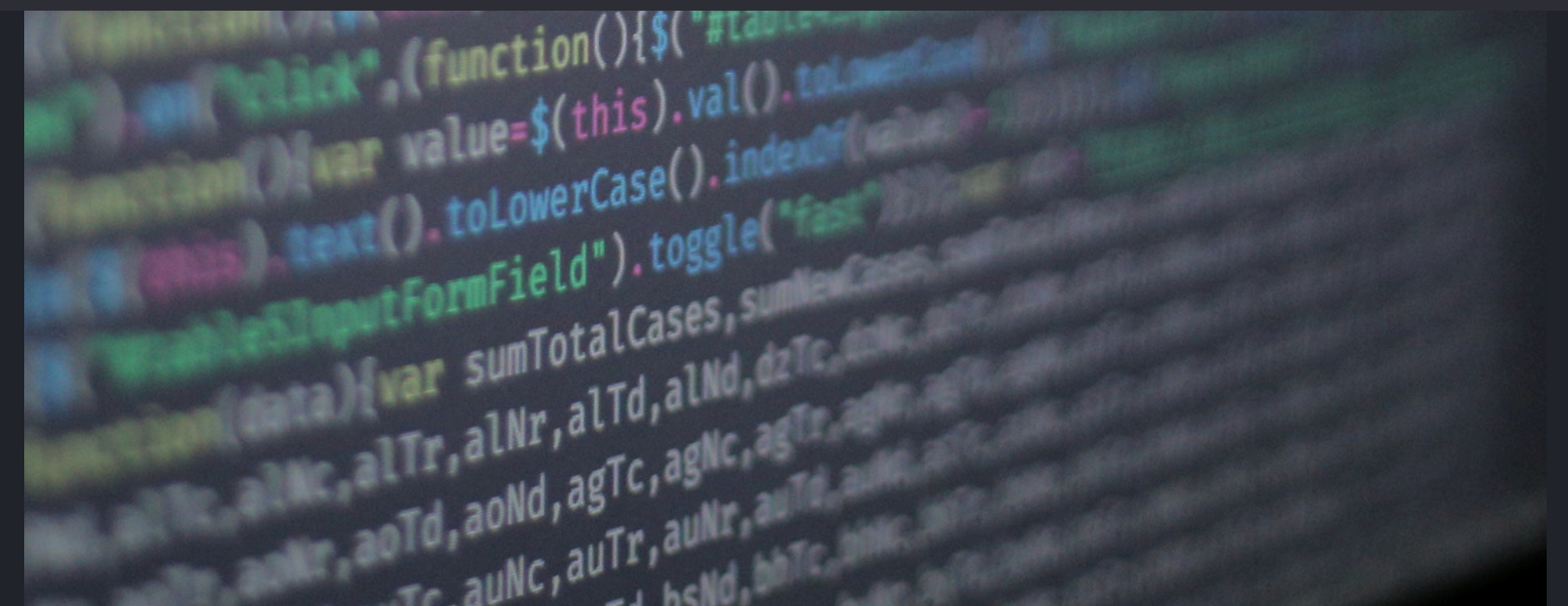
while True:
    intento = math.floor((inferior + superior) / 2)
    print(f"\nLa maquina adivina: {intento}")
    print("1 = Si, acertaste")
    print("2 = El numero es menor")
    print("3 = El numero es mayor")

    respuesta = int(input("Tu respuesta: "))
    intentos += 1

    if respuesta == 1:
        print(f"\n¡Acerte en {intentos} intentos!")
        break
    elif respuesta == 2:
        superior = intento - 1
    elif respuesta == 3:
        inferior = intento + 1
```

La parte mas dificil al desarrollar proyecto

AL DESARROLLAR EL PROYECTO LA MAYOR DIFICULTAD FUE JUSTO EN LA OPERACION CLAVE PARA HALLAR $((\text{INFERIOR} + \text{SUPERIOR}) / 2)$, AUNQUE ESTE BIEN PLANTEADA AL MOMENTO DE REALIZAR LA OPERACION SE CONVERTIA EN DECIMAL, SIM EMBARGO BUSCANDO LA SOLUCION HALLE LA FUNCION MATEMATICA MATH LA CUAL AYUDA A DAR NUMEROS ENTEROS



Ejemplo de ejecucion

Si el usuario piensa el numero 73, la maquina lo adivina en 4 intentos:

- Intento 1: $\text{math.floor}((1+100)/2) = 50 \rightarrow$ usuario responde 3 (mayor)
- Intento 2: $\text{math.floor}((51+100)/2) = 75 \rightarrow$ usuario responde 2 (menor)
- Intento 3: $\text{math.floor}((51+74)/2) = 62 \rightarrow$ usuario responde 3 (mayor)
- Intento 4: $\text{math.floor}((63+74)/2) = 68 \rightarrow \dots$ hasta llegar a 73

En el peor caso, la maquina necesita 7 intentos para adivinar cualquier numero entre 1 y 100.

}

Reflexion sobre el Impacto de la Tecnologia

El desarrollo de este proyecto permite reflexionar sobre como la logica computacional puede resolver problemas de forma mas eficiente que el pensamiento humano intuitivo. Mientras una persona intentando adivinar un numero podria hacerlo al azar o de forma secuencial, el computador aplica un algoritmo matematicamente optimo que garantiza encontrar la respuesta en un numero logaritmico de pasos.

Este principio de busqueda binaria no es solo un ejercicio academico: es la base de algoritmos utilizados en motores de busqueda, bases de datos, sistemas de ordenamiento y miles de aplicaciones del mundo real.

Comprender y programar este tipo de logica desde la asignatura de Logica de Programacion sienta las bases para entender como funciona el software que usamos cotidianamente.

Ademas, el proyecto demuestra que la tecnologia no reemplaza al humano sino que lo complementa: en este juego, el computador necesita la respuesta del usuario para funcionar, estableciendo una interaccion entre la inteligencia humana y la capacidad de calculo de la maquina. Esta colaboracion es el fundamento de los sistemas interactivos modernos.

Finalmente, el ejercicio de traducir un proceso de pensamiento logico (como adivinar un numero) a un programa funcional en Python refuerza una habilidad clave del siglo XXI: el pensamiento computacional, es decir la capacidad de descomponer problemas complejos en pasos simples, sistematicos y ejecutables por una maquina.

}

Ing. Ciberseguridad

Gracias {

}